

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a49721afeb2d618e7.jsonl`  
- SHA-256: `802233b2e33441113014605b49d333368220057d573326b886fd0370aacfa8e7`  
- Source modified: `2026-07-02T03:02:57+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T02:58:05.137Z)

You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.

A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer:

```
{  
  "title": "Import cycles: extracted API modules reach back into backend.main globals at call time; config layer imports pipeline layer",  
  "category": "tech-debt",  
  "severity": "P1",  
  "file": "backend/api/job_worker.py",  
  "line": 75,  
  "evidence": "AST scan of backend/ found 5 strongly-connected components. (1) backend.main <-> backend.api.job_worker <-> backend.api.uploads: job_worker.py:75,98,214,234,251 and uploads.py:47 do call-time 'import backend.main as _main' to read main.py's module globals db_instance/global_config (main.py:279-280); main.py:292 re-exports job_worker names back onto itself. Documented as a transitional shim (CODE_CLEANUP_AUDIT P5 note, line 221) but it makes main.py load-bearing for every extracted module and keeps backend.main AND backend.worker.__main__ mypy-quarantined (mypy.ini:15-23). (2) Layering inversion: backend/config/models.py:249 imports segmenter_registry from backend.pipeline.segmenters.base inside a validator, while base.py:15 (_normalize_config) imports AppConfig back from config.models ? config validation now depends on pipeline registration side effects. (3-5) eval-package cycles (golden_fixtures<->golden_real_fixtures, ablation<->ablation_pdf, annotations<->gt_loader) are lazy and explicitly documented (golden_fixtures.py:1005-1028) ? lower priority.",  
  "proposed_fix": "Kill the shim as the concrete step-2 of D1: move db_instance/global_config onto app.state (create_app already exists per P5) and pass config/db explicitly to job_worker/uploads, then lift backend.main and backend.worker.__main__ out of mypy quarantine. For the layering inversion, move the segmenter-name validation out of the pydantic validator into a startup check (backend/config/startup.py) so config/ no longer imports pipeline/.",  
  "effort": "M",  
  "tracked_in": "docs/CODE_CLEANUP_AUDIT_2026-06-24.md P5 note + D1/P10 follow-up (line 221)"  
}
```

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator, C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field; non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup', C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).

VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it

owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training, product/strategy/licensing, real cloud spend, counsel ToS/DPA?
HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you personally saw the defect/debt in current code; if you cannot reproduce the claim from the code, say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T02:58:17.313Z)

Already up to date.

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
aa915a7 deploy/gcp: centralize project + bucket names in one sourced buckets.env (#410)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)

3. user (2026-07-02T02:58:22.870Z)

```
60         The wiring is additive: the production segmenter path never raises this today (zero
61         behaviour change), so a job runs exactly as before. The hook is what a later slice
62         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
63         """
64
65         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
66             super().__init__(f"job parked for {delay_sec:.0f}s")
67             self.delay_sec = max(0.0, float(delay_sec))
68             self.progress = progress
69
70
71         def _job_to_request(job: Dict[str, Any]):
72             """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
73             re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
74             the original submit. The row stores exactly the ProcessRequest fields."""
75             import backend.main as _main
76
77             return _main.ProcessRequest(
78                 video_path=job["video_path"],
79                 player_pool=job.get("player_pool"),
80                 max_frames=job.get("max_frames"),
81                 segmenter=job.get("segmenter"),
82                 # v8 compute-picker: the worker runs _execute_processing ?
83                 _maybe_dispatch_remote, which
84                 # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
85                 # the job row for analytics). NULL ? server default = pre-picker behaviour.
86                 compute=job.get("compute"),
87             )
88
89         def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
90             """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
91             its
92             terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
93             Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
94             means
95             the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
96             result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
97             the
98             job self-scheduled and will be re-claimed when `next_poll_at` is due.
99             """
100             import backend.main as _main
101
102             job_id = job["id"]
103             # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
```

```

(NULL/default ? the
102     # original /api/process pipeline). Both record their terminal state the same way;
only process
103     # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
104     is_compile = job.get("kind") == "compile"
105     try:
106         if is_compile:
107             result = _main._execute_compile(
108                 config,
109                 db,

```

4. user (2026-07-02T02:58:23.663Z)

```

265
266
267     def _bundled_ui_version() -> Optional[Dict[str, Any]]:
268         """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
bundled.
269         Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
guard)."""
270         path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
271         try:
272             with open(path, "r", encoding="utf-8") as f:
273                 return json.load(f)
274         except (OSError, ValueError):
275             return None
276
277
278     # Global variables initialized at startup
279     db_instance: Optional[Database] = None
280     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
281     # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
282     # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
283     _instance_lock: Optional[Any] = None
284
285     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
286     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
287     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
288     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
289     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
290     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
291     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
292     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
293         _MAX_DRAIN_WAKE_SEC,
294         JobWaiting,
295         _arm_wake_timer,
296         _drain_due_jobs,
297         _get_executor,
298         _job_to_request,
299         _maybe_record_job_cost,
300         _run_claimed_job,
301         _schedule_next_drain_if_waiting,
302     )
303
304

```

5. user (2026-07-02T02:58:30.205Z)

badminton-highlight-indexer\backend\api\job_worker.py:11: `backend.main` module object at call-

```

time (`import backend.main as _main`), never as a
badminton-highlight-indexer\backend\api\job_worker.py:75: import backend.main as _main
badminton-highlight-indexer\backend\api\job_worker.py:98: import backend.main as _main
badminton-highlight-indexer\backend\api\job_worker.py:214: import backend.main as _main
badminton-highlight-indexer\backend\api\job_worker.py:234: import backend.main as _main
badminton-highlight-indexer\backend\api\job_worker.py:251: import backend.main as _main
badminton-highlight-indexer\backend\api\uploads.py:47: import backend.main as _main

```

6. user (2026-07-02T02:58:30.928Z)

```

35     filename: str
36     total_size_bytes: Optional[int] = None
37
38
39     class UploadCompleteRequest(BaseModel):
40         total_parts: int
41
42
43     def _upload_cfg():
44         # Resolved lazily (import inside the function) to read the LIVE startup config that
45         # main() sets on the module global, and to avoid a circular import at module load
46         # (main.py imports this router; this module must not import main at import time).
47         import backend.main as _main
48
49         sec = getattr(_main.global_config, "security", None)
50         upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
51         max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
52         part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
53         exts = [
54             e.lower().lstrip(".")
55             for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
56         ]
57         return upload_dir, max_bytes, part_bytes, exts
58
59
60     def _abort_upload(upload_id: str) -> None:
61         with _uploads_lock:
62             st = _uploads.pop(upload_id, None)
63             if st:
64                 try:

```

7. user (2026-07-02T02:58:36.841Z)

```

235     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
236     skip_ai_handoff: bool = False
237     # Optional debug knob: trim every video to the first N seconds before processing.
238     debug_duration: Optional[float] = None
239
240     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
241     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
242     fusion: FusionConfig = Field(default_factory=FusionConfig)
243
244     @field_validator("default_segmenter")
245     @classmethod
246     def segmenter_must_be_registered(cls, v: str) -> str:
247         # Lazy import: the segmenter registry imports modules that may import
248         # config, so we resolve it at validation time rather than at module load.
249         from backend.pipeline.segmenters.base import segmenter_registry
250
251         if segmenter_registry.get(v) is None:
252             available = sorted(segmenter_registry.list_available().keys())
253             raise ValueError(
254                 f"Segmenter '{v}' is not registered. Available: {' ', ' '.join(available)}"
255             )

```

```

256         return v
257
258     @model_validator(mode="after")
259     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
260         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
261         # step makes `while t < duration: t += step` loop forever, growing memory before
any
262         # GPU work. Reject the bad config at load time instead.
263         if self.chunk_overlap >= self.chunk_duration:
264             raise ValueError(
265                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
266                 f"({self.chunk_duration}); otherwise chunking never advances."
267             )
268         return self
269

```

8. user (2026-07-02T02:58:37.830Z)

```

1     from abc import ABC, abstractmethod
2     from typing import Any, Dict, List, Optional, Type
3
4     from backend.infrastructure.database import Database
5     from backend.results import (
6         Result, # new standardized contract (Result[Value] == Success[Value] | Failure)
7     )
8
9
10    def _normalize_config(config: Any) -> Any:
11        """Normalize a plain-dict config into an AppConfig so segmenters can use typed
12        attribute access consistently. Production always passes AppConfig; tests may
13        pass plain dicts ? this bridge keeps both paths working."""
14        if isinstance(config, dict):
15            from backend.config.models import AppConfig
16
17            try:
18                return AppConfig(**config)
19            except Exception:
20                return (
21                    config # invalid config ? let the segmenter's own validation reject it
22                )
23        return config
24
25
26    class BasePlaySegmenter(ABC):
27        def __init__(self, db: Database, config: Dict[str, Any]):
28            self.db = db
29            self.config = _normalize_config(config)
30
31        @property
32        @abstractmethod
33        def name(self) -> str:
34            """Unique identifier for this segmenter."""
35            pass
36
37        @property
38        @abstractmethod
39        def description(self) -> str:
40            """Human-readable explanation of this segmenter."""
41            pass
42
43        @abstractmethod
44        def process_video(
45            self,

```

```

46         video_id: str,
47         video_path: str,
48         player_pool: List[str] = None,
49         max_frames: int = None,
50     ) -> "Result[List[Dict[str, Any]]]":
51         """
52         Processes a video to detect play segments, populates SQLite database,
53         and returns a Result.
54
55         Use isinstance(result, Failure) (or result.is_success()) to distinguish
56         "no segments found" from "something went wrong" (missing key, healthcheck
57         failure, etc.). This is the start of the standardized error/result contract.
58         """
59         pass
60

```

9. user (2026-07-02T02:58:45.560Z)

```

1     [mypy]
2     # Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean
3     # modules and pins an explicit quarantine list below ? not a typing crusade.
4     # Ratchet: when you touch a quarantined module, try removing its entry.
5     ignore_missing_imports = True
6     # backend/ is a namespace package (no root __init__.py) ? required so mypy
7     # maps files to backend.* module names instead of duplicating top-level names.
8     explicit_package_bases = True
9
10    # --- Quarantine (known-loose modules, each with a reason) ---
11
12    # Optional module globals (db_instance/global_config set by create_app() / main()).
13    # Route handlers use request.app.state exclusively; helpers take explicit params.
14    # backend.api.job_worker lifted from quarantine (now takes explicit config/db params).
15    [mypy-backend.main]
16    ignore_errors = True
17
18    # Worker dispatcher: composes the SAME validator/segmenter/report building blocks as
19    # backend.main, which are typed `config: dict` but receive the AppConfig migration shim
20    # at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine
21    # until the typed-config migration removes the dict?AppConfig duality everywhere.
22    [mypy-backend.worker.__main__]
23    ignore_errors = True
24
25    # yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code
26    # to detectors/person_detector.py (which IS type-clean, no quarantine). What remains
27    # here is pre-existing orchestration debt unrelated to that move (implicit-Optional
28    # process_video defaults, the Success|Failure ABC-override mismatch, a Future[...]
29    # annotation) ? ratchet those out in a focused typing pass, not here.
30    # (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)

```

10. user (2026-07-02T02:58:46.889Z)

```

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)
7ea6ea9 P3: /api/capabilities reports usable storage backends (#408)
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
759a67f feat(api): frame-preserving annotation proxy for /api/source (D4) (#406)
5a5c4fa feat(api): P8a ? GET /api/source + POST /api/annotations (annotate-in-UI backend) (#405)
959c630 fix(#397): write per-video report to the resolved local path, not the raw URI (#400)
39dff1d feat(config): promote inactivity_temporal_density to default-ON (P2b) (#396)
cc60ebe Fix/rally detection edge cases (#394)

```

11. user (2026-07-02T02:58:57.954Z)

```

200         flywheel.run_for_job(job, result, config)
201     except Exception as e: # never wedge a completed job on the flywheel
202         logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
203
204
205     def _drain_due_jobs(config: Any, db: Any) -> int:
206         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
207         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
208
209         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
210         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
211         a single-worker box keeps making progress with no central timer (the DB is the
clock).
212         Returns the number of jobs that reached a terminal/parked state this tick.
213         """
214         import backend.main as _main
215
216         ran = 0
217         while True:
218             job = db.claim_due_job()
219             if job is None:
220                 break
221             ran += 1
222             _main._run_claimed_job(job, config, db)
223             # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
224             # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
225             # on the next submit/drain. Best-effort ? never raises into the worker.
226             _main._schedule_next_drain_if_waiting(config, db)
227             return ran
228
229
230     def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
231         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
232         self-scheduled job resumes with no further client submit. The drain itself re-checks
233         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
234         import backend.main as _main
235
236         try:
237             delay = db.seconds_until_next_due()
238         except Exception as e: # pragma: no cover - defensive; never wedge the worker
239             logger.error(f"Could not compute next-due delay: {e}")
240             return
241         if delay is None:
242             return # nothing waiting
243         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
244         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
245
246
247     def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
248         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
249         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
250         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
251         import backend.main as _main
252
253         timer = threading.Timer(

```

```

254         delay_sec,
255         lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
256     )
257     timer.daemon = True
258     timer.start()
259
260
261     #: A parked job further out than this is re-checked by a capped wake rather than one
long
262     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
263     _MAX_DRAIN_WAKE_SEC = 60.0
264

```

12. user (2026-07-02T02:58:58.737Z)

```

1     """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3     Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4     The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5     that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6     re-claimed when due ? with no central scheduler (the DB is the clock).
7
8     The monkeypatch contract these functions rely on (and that tests depend on):
9     every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11    `backend.main` module object at call-time (`import backend.main as _main`), never as a
12    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13    ...) ` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15    inside the functions also avoids the circular import at module load (main.py imports
this
16    module; this module must not import main at import time).
17    """
18
19    import logging
20    import threading
21    import traceback
22    from concurrent.futures import ThreadPoolExecutor
23    from typing import Any, Dict, Optional
24
25    logger = logging.getLogger(__name__)
26
27    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
28    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
29    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
30    # already parallelize their AI handoff internally via their own pools.
31    #
32    # ? 02 RISK (CODE_CLEANUP_AUDIT $3c): a single worker means ONE stuck or hung job
33    # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
34    # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
35    #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
36    #   - detector timeout_sec kills hung WSL/GPU calls
37    #   - Cloud Run poll has a max_wait_sec bound
38    # A future slice should add a per-job wall-clock timeout (the executor itself
39    # cannot enforce timeouts on the Thread).
40    _job_executor: Optional[ThreadPoolExecutor] = None

```

13. user (2026-07-02T02:59:08.904Z)

```

77:_main.ProcessRequest
107:_main._execute_compile
114:_main._job_to_request
115:_main._execute_processing

```



```
131:_main.JobWaiting
222:_main._run_claimed_job
226:_main._schedule_next_drain_if_waiting
244:_main._arm_wake_timer
244:_main._MAX_DRAIN_WAKE_SEC
255:_main._get_executor
255:_main._drain_due_jobs
```

14. user (2026-07-02T02:59:09.938Z)

```
49:_main.global_config
```

15. user (2026-07-02T02:59:21.978Z)

```
badminton-highlight-indexer\backend\main.py:98: Also sets the module-level ``global_config``
/ ``db_instance`` for transitional
badminton-highlight-indexer\backend\main.py:101: global global_config, db_instance
badminton-highlight-indexer\backend\main.py:102: global_config = config
badminton-highlight-indexer\backend\main.py:103: db_instance = db
badminton-highlight-indexer\backend\main.py:279:db_instance: Optional[Database] = None
badminton-highlight-indexer\backend\main.py:280:global_config: Optional[AppConfig] = None # now
typed (Pydantic model)
badminton-highlight-indexer\backend\main.py:1138: module-level
``global_config``/``db_instance`` (backward-compatible with existing
badminton-highlight-indexer\backend\main.py:1157: config = global_config
badminton-highlight-indexer\backend\main.py:1159: db = db_instance
badminton-highlight-indexer\backend\main.py:1482:# backend.main.global_config, so behavior ?
route paths, status codes, body-size limits,
badminton-highlight-indexer\backend\main.py:1938: # global_config isn't initialized this
early; load the typed config so the trim
badminton-highlight-indexer\backend\main.py:1981: global global_config, db_instance
badminton-highlight-indexer\backend\main.py:1982: global_config = load_app_config() #
returns typed AppConfig
badminton-highlight-indexer\backend\main.py:1984: global_config
badminton-highlight-indexer\backend\main.py:1992: global_config.output.output_dir =
resolve_output_dir(args.output_dir)
badminton-highlight-indexer\backend\main.py:1993:
os.makedirs(global_config.output.output_dir, exist_ok=True)
badminton-highlight-indexer\backend\main.py:1996: global_config.output.output_dir,
global_config.output.db_name
badminton-highlight-indexer\backend\main.py:2026: db_instance = Database(db_path)
badminton-highlight-indexer\backend\main.py:2030: swept = db_instance.fail_stale_jobs()
badminton-highlight-indexer\backend\main.py:2039: _maybe_sweep_gemini_orphans(global_config)
badminton-highlight-indexer\backend\main.py:2047: storage_cfg = getattr(global_config,
"storage", None)
badminton-highlight-indexer\backend\main.py:2059: was_reingest =
db_instance.get_video(video_id) is not None
badminton-highlight-indexer\backend\main.py:2064: local_video, global_config
badminton-highlight-indexer\backend\main.py:2070: db_instance.add_video(
badminton-highlight-indexer\backend\main.py:2075: getattr(global_config, "indexing",
None), "default_segmenter", "motion"
badminton-highlight-indexer\backend\main.py:2100: play_wm, cand_wm =
db_instance.segment_watermarks(video_id)
badminton-highlight-indexer\backend\main.py:2101: segmenter =
segmenter_cls(db_instance, global_config)
badminton-highlight-indexer\backend\main.py:2117: _finalize_reingest(db_instance,
video_id, segments, play_wm, cand_wm)
badminton-highlight-indexer\backend\main.py:2121: db=db_instance,
badminton-highlight-indexer\backend\main.py:2127: config=global_config,
badminton-highlight-indexer\backend\main.py:2165: global_config, args.host
badminton-highlight-indexer\backend\main.py:2181: _get_executor().submit(_drain_due_jobs,
global_config, db_instance)
badminton-highlight-indexer\backend\main.py:2184: create_app(global_config, db_instance)
badminton-highlight-indexer\backend\main.py:2190: # shared mutable state (`db_instance`,
`global_config`) via `import backend.main` ? a SEPARATE
```

```
badminton-highlight-indexer\backend\main.py:2192:    # `backend.main.db_instance` = None; the
async-job drain then crashes silently on
badminton-highlight-indexer\backend\api\job_worker.py:9:every cross-function seam ?
`db_instance`, `global_config`, `_execute_processing`,
badminton-highlight-indexer\backend\api\uploads.py:10:The startup config lives on
`backend.main.global_config` (set ONLY in `main()`).
badminton-highlight-indexer\backend\api\uploads.py:13:`backend.main.global_config` flow
through unchanged).
badminton-highlight-indexer\backend\api\uploads.py:49:    sec = getattr(_main.global_config,
"security", None)
```

16. user (2026-07-02T02:59:23.665Z)

1. **Q: Should D1 (split `main.py`) be Phase 1 or Phase 2? Leaning Phase 2: D2 (app-factory) is a natural precursor, and D1 is lower risk after the globals are gone. Owner to confirm.
2. ~~Q: Should Phase 1 include enabling `I` (isort) in ruff.toml?~~ ? ? Shipped in #360.

8. Definition of done

- [x] P0: This doc approved and merged to `master`
- [x] P1?P3: Typed-config migration complete ? no `.get()` dict access remains in backend code; `extra='forbid'` on `AppConfig`
- [x] P4: `segmenter\_must\_be\_registered` actually validates against the registry
- [x] P5: `create\_app()` injects shared state via `app.state` (handlers read `request.app.state`); `backend.api.job\_worker` removed from mypy quarantine. (The Optional module globals `db\_instance`/`global\_config` are **retained as a transitional back-compat shim** and `backend.main` stays mypy-quarantined ? full removal is follow-up, see D1/P10.)
- [x] P6: Quick wins Q1?Q6, Q8 landed
- [x] P7: Full test suite ? 1463/1463 pass (#371); `ruff` + `mypy` clean; `backend.api.job\_worker` lifted from quarantine
- [x] All PRs are single-purpose, branched from `master`, with descriptive commit messages
- [] `docs/README.md` updated to reflect the project's terminal state
- [] Final status flipped to `DONE` and this doc moved to `docs/archives/past\_projects/`

8.1 ? Next items ? Phase 2 pick-up (prioritized)

All items below are **Claude-doable**, small-PR-friendly, and unblocked. Pick up in order:

Immediate (HIGH impact, low risk)

17. user (2026-07-02T02:59:50.317Z)

```
2180         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
2181         _get_executor().submit(_drain_due_jobs, global_config, db_instance)
2182         # Stamp shared state on the app so route handlers access it via
request.app.state
2183         # instead of the Optional module-level globals (CODE_CLEANUP_AUDIT P5 / D2).
2184         create_app(global_config, db_instance)
2185         uvicorn.run(app, host=bind_host, port=port)
2186
2187
2188     if __name__ == "__main__":
2189         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
2190         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
2191         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
2192         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
2193         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
```

```

through
2194         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
2195         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
2196         import backend.main as _bm
2197
2198         _bm.main()
2199

```

18. user (2026-07-02T02:59:51.324Z)

```

badminton-highlight-indexer\backend\pipeline\segmenters\base.py:66:     def register(self,
segmenter_cls: Type[BasePlaySegmenter]):
badminton-highlight-indexer\backend\pipeline\segmenters\base.py:93:segmenter_registry =
SegmenterRegistry()
badminton-highlight-indexer\backend\pipeline\segmenters\fusion_hybrid.py:37:from
backend.pipeline.segmenters.base import segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\fusion_hybrid.py:241:segmenter_registry.
register(FusionSegmenter)
badminton-highlight-indexer\backend\pipeline\segmenters\tracknet_hybrid.py:19:from
backend.pipeline.segmenters.base import segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\tracknet_hybrid.py:48:segmenter_registry
.register(TrackNetHybridSegmenter)
badminton-highlight-indexer\backend\pipeline\segmenters\motion.py:8:from
backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\motion.py:243:segmenter_registry.registe
r(MotionPlaySegmenter)
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:8:from
backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:694:segmenter_registry.registe
r(GeminiPlaySegmenter)
badminton-highlight-indexer\backend\pipeline\segmenters\wasb_hybrid.py:21:from
backend.pipeline.segmenters.base import segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\wasb_hybrid.py:64:segmenter_registry.reg
ister(WasbHybridSegmenter)
badminton-highlight-indexer\backend\pipeline\segmenters\yolo_hybrid.py:26:from
backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\yolo_hybrid.py:558:segmenter_registry.re
gister(HybridYoloSegmenter)
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:1:from
backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:10:     "segmenter_registry",

```

19. user (2026-07-02T03:00:00.372Z)

```

badminton-highlight-indexer\backend\main.py:63:from backend.pipeline.segmenters.base import
segmenter_registry
badminton-highlight-indexer\backend\worker\__main__.py:38:from backend.pipeline.segmenters.base
import segmenter_registry
badminton-highlight-indexer\backend\worker\__main__.py:423:     from
backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
badminton-highlight-indexer\backend\features\__init__.py:4:imported by BOTH the training harness
(training/) and the serving segmenters, so
badminton-highlight-indexer\backend\config\models.py:249:     from
backend.pipeline.segmenters.base import segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\fusion_hybrid.py:37:from
backend.pipeline.segmenters.base import segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\fusion_hybrid.py:38:from
backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:8:from
backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:105:     from
backend.pipeline.segmenters._keyhint import api_key_hint
badminton-highlight-indexer\backend\pipeline\segmenters\tracknet_hybrid.py:19:from

```

```

backend.pipeline.segmenters.base import segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\tracknet_hybrid.py:20:from
backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\motion.py:8:from
backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\yolo_hybrid.py:26:from
backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\yolo_hybrid.py:84:         from
backend.pipeline.segmenters._keyhint import api_key_hint
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:1:from
backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:2:from
backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:3:from
backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:4:from
backend.pipeline.segmenters.motion import MotionPlaySegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:5:from
backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:6:from
backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:7:from
backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\wasb_hybrid.py:21:from
backend.pipeline.segmenters.base import segmenter_registry
badminton-highlight-indexer\backend\pipeline\segmenters\wasb_hybrid.py:22:from
backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\trajectory_hybrid.py:31:from
backend.pipeline.segmenters.base import BasePlaySegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\trajectory_hybrid.py:206:
from backend.pipeline.segmenters._keyhint import api_key_hint
badminton-highlight-indexer\backend\eval\served_gate_a.py:35:from
backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter

```

20. user (2026-07-02T03:00:01.303Z)

```

    Use isinstance(result, Failure) (or result.is_success()) to distinguish
    "no segments found" from "something went wrong" (missing key, healthcheck
    failure, etc.). This is the start of the standardized error/result contract.
    """
    pass

```

```

class SegmenterRegistry:
    def __init__(self):
        self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}

    def register(self, segmenter_cls: Type[BasePlaySegmenter]):
        """Registers a new segmenter class under its name property."""
        try:
            temp_inst = segmenter_cls(None, {})
            name = temp_inst.name
        except Exception as e:
            raise ValueError(
                f"Failed to resolve name property from segmenter class {segmenter_cls.__name__}:
{e}"
            ) from e
        self._segmenters[name] = segmenter_cls

    def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
        return self._segmenters.get(name)

    def list_available(self) -> Dict[str, str]:
        """Returns a dict of registered segmenter names and their descriptions."""
        res = {}

```

```

    for name, cls in self._segmenters.items():
        try:
            temp_inst = cls(None, {})
            desc = temp_inst.description
        except Exception:
            desc = "No description available."
        res[name] = desc
    return res

```

```
segmenter_registry = SegmenterRegistry()
```

21. user (2026-07-02T03:00:20.589Z)

```

25:from backend.utils.app_home import env_file_to_load, resolve_output_dir
45:from backend import (
49:from backend.api import (
52:from backend.config import (
59:from backend.config import ( # new typed config
62:from backend.infrastructure.database import Database
63:from backend.pipeline.segmenters.base import segmenter_registry
64:from backend.pipeline.stitcher.compiler import VideoCompiler
65:from backend.pipeline.stitcher.watermark_i18n import localize_watermark
66:from backend.pipeline.validators.base import registry
67:from backend.providers import provider_registry
68:from backend.reporting import emit_indexer_report
69:from backend.results import Failure # standardized error/result contract
70:from backend.storage.capabilities import (
73:from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
77:from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
78:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
79:from backend.utils.paths import (
85:from backend.utils.redact import (
88:from backend.utils.single_instance import acquire_lock # P0e: one instance per database
292:from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable on
backend.main)
1484:from backend.api import uploads as uploads_api

```

22. user (2026-07-02T03:00:27.489Z)

```

from backend import (
    billing, # M8: per-job cost ledger (USD internal / INR display)
    storage, # M2: URI-aware input acquisition (local = identity passthrough)
)
from backend.api import (
    auth as edge_auth, # M9: Cloudflare Access edge-auth (default-OFF)
)
from backend.config import (
grep: backend/__init__.py: No such file or directory

```

23. user (2026-07-02T03:00:36.823Z)

```

63:from backend.pipeline.segmenters.base import segmenter_registry
452:    "segmenters": sorted(segmenter_registry.list_available().keys()),
882:    persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters call ?
so
1224:    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
1407:    detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
1876:    available_segmenters = list(segmenter_registry.list_available().keys())
1879:    choices=available_segmenters,
1880:    help=f"Choose rally detection segmenter. Available: {available_segmenters}",

```

24. user (2026-07-02T03:00:47.461Z)

No matches found

25. user (2026-07-02T03:00:54.909Z)

```
badminton-highlight-indexer\backend\eval\served_gate_a.py:35:from
backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:2:from
backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:3:from
backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:4:from
backend.pipeline.segmenters.motion import MotionPlaySegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:5:from
backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:6:from
backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py:7:from
backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
```

26. user (2026-07-02T03:01:13.295Z)

is exposed here so every historical ``golden\_fixtures.<name>`` import path stays valid

```
=====

#: The names the real-fixture module owns; re-exported lazily (PEP 562 ``__getattr__``) so the
#: ``golden_fixtures`` ? ``golden_real_fixtures`` cycle resolves regardless of import order ?
#: ``golden_real_fixtures`` imports the shared primitives FROM this module, so an EAGER bottom-
of-
#: file re-export would fail when ``golden_real_fixtures`` is imported FIRST (its own names
would
#: not yet be defined when this module's import of them runs). The lazy hook defers the import
to
#: first attribute access, by which point BOTH modules are fully initialized.
_REAL_FIXTURE_EXPORTS: Tuple[str, ...] = (
    "RefusalError",
    "freeze_real_fixture",
    "_scan_forbidden",
    "_new_surrogate_slug",
    "_reset_trajectory_rows",
    "_reset_labels",
    "_SCHEMA_FILENAME_LITERALS",
)

def __getattr__(name: str) -> Any:
    """Lazily resolve the re-exported real-fixture names from ``golden_real_fixtures`` (PEP
562)."""
    if name in _REAL_FIXTURE_EXPORTS:
        from backend.eval import golden_real_fixtures

        return getattr(golden_real_fixtures, name)
150:     default_segementer: str = "yolo_hybrid"
244:     @field_validator("default_segementer")
```

27. user (2026-07-02T03:01:50.273Z)

No files found

28. user (2026-07-02T03:01:52.085Z)

40:| ****D1**** | ****Phase 1 = pure refactorings only.**** No logic or behavior change. All tests must

pass; coverage must not decrease. | Owner, 2026-06-24 | B2 (Success/Failure contract for Gemini) is deferred to Phase 2 since it changes return types callers depend on. D3 (typed-config migration), D2 (app-factory), and B5 (segmenter validator) are pure-refactoring-possible and are the first targets. |

46:| ****D7**** | ****Absorb relevant BACKLOG items.**** Deferred hardening items from `docs/BACKLOG.md` that overlap with this audit are absorbed into this project rather than left in BACKLOG. Items that are feature-gaps (not code-cleanup) stay in BACKLOG. | PR #352 review, 2026-06-24 | D2 and O1 were already independently found; D13 and D14 are new absorptions. The train/eval splits guardrail stays in BACKLOG (it needs a collector-side manifest change). |

69:| ****D1**** | HIGH | `backend/main.py` | ****Massive god-module**** (~1450 lines). CLI, API, worker orchestration, upload, debug-clip, trim, reingest all in one file. | 2 |

78:| ****D10**** | LOW | `backend/reporting/spec.yaml` + `harvest.py` | Tight coupling via magic `when.path` strings. Rename a detail ? rule silently stops firing. | 3 |

79:| ****D11**** | LOW | `backend/pipeline/stitcher/compiler.py` | Two-stage codec contract is implicit. Change a preset ? concat breaks. | 3 |

80:| ****D12**** | LOW | `tools/generate\_highlights.py` | Uses `sys.path.insert(0, ...)` hack instead of proper package install. | 3 |

81:| ****D13**** | MEDIUM | `requirements.txt`, `pyproject.toml` | ****Absorbed from BACKLOG.**** `supervision` ByteTrack is deprecated as of 0.28.0 and ****removed in 0.30.0****. The `<0.30.0` pin in `requirements.txt` and `pyproject.toml` is a stopgap ? migration to the standalone `trackers` package is needed before the pin can be lifted. | 2 |

82:| ****D14**** | LOW | `tests/` directory | ****Absorbed from BACKLOG.**** 90+ test files in a flat `tests/` directory. Owner wants reorganization by subsystem (e.g. `tests/eval/`, `tests/audio/`) to reduce fragmentation. The exact grouping metric is the owner's design choice. | 3 |

113:| ****D2**** | "App-factory / lifespan + lazy torch-cv2 imports" | ? ? "Belongs with the async-job slice (#16)" ? #16 is now done; unblocked | Phase-1 target (P5) |

115:| ****D13**** | "supervision` ByteTrack deprecation" | ? ? pinned `<0.30.0` as stopgap; migration to `trackers` package needed | New finding in this doc (see §3b) |

116:| ****D14**** | "`tests/` reorganization to reduce fragmentation" | ? ? owner's design choice on grouping metric needed first | New finding in this doc (see §3b) |

168:- ****Does NOT split `main.py` (D1).**** The god-module split is deferred ? D2 (app-factory) is a necessary precursor.

- [x] P5: `create\_app()` injects shared state via `app.state` (handlers read `request.app.state`); `backend.api.job\_worker` removed from mypy quarantine. (The Optional module globals `db\_instance`/`global\_config` are ****retained as a transitional back-compat shim**** and `backend.main` stays mypy-quarantined ? full removal is follow-up, see D1/P10.)

29. user (2026-07-02T03:02:01.884Z)

```
__init__.py
__pycache__
loader.py
models.py
presets.py
startup.py
```

30. user (2026-07-02T03:02:08.783Z)

""Startup checks ? per-profile coherence (M5) + exposure-safety posture (M9 hardening).

A deployment profile (`truly-local` / `cloud-serving` / `cpu-mock`) implies a coherent runtime. These checks ****fail/warn fast at process startup**** ? a loud signal *before* any (expensive) job runs ? when the active profile is inconsistent with the environment:

- * ****cloud-serving with a non-cloud storage backend**** ? a hard ****error**** (the one config bug we can detect with certainty + zero false-positive risk: a cloud profile that can't reach a bucket can never serve). This is the README §4.3 "loud error before any job".
- * a ****GPU mismatch**** (e.g. ``device='cuda'`` on a box where CUDA isn't visible, or ``device='auto'`` falling back to CPU) ? a ****warning****, never an error: the probe is best-effort (torch may live only in the detector subprocess env ? see ``presets.probe\_cuda\_available``), and the detector's own healthcheck is the real authority. A warning surfaces it early without false-failing.
- * ****creds that can't be confirmed**** (no ``GOOGLE\_APPLICATION\_CREDENTIALS`` / not on Cloud Run)

?
a ****warning****: Application Default Credentials can come from the metadata server / gcloud,
which
we can't probe offline, so the storage backend's own init is the authority; we just flag it.

****Exposure-safety posture (server only, ``include\_exposure=True``).**** When the engine is
exposed
behind an edge gate (the owner flips ``security.edge\_auth\_enabled`` on for the Cloudflare-Access
boundary ? the alpha-shareable "tunnel-to-a-box" topology), the gate is only half the safety
story.
A half-configured exposure starts **looking** healthy then leaks / 500s per request, so we verify
the
rest of the posture at ****boot****: the ``allowed\_video\_root`` path-jail is set (else an
authenticated
tenant could read an arbitrary local file ? ****error****), the CF team-domain + AUD are present
(else
no token can verify ? ****error****, lifting the current first-request failure to boot), and the
``auth`` extra is importable (else every request 500s ? ****warning****). Only the HTTP ****server****
passes

31. user (2026-07-02T03:02:57.761Z)

Structured output provided successfully